

MÔN HỌC

DEEP LEARNING - HỌC SÂU

HỌC KỲ 2533 NĂM HỌC 2025 - 2026

ĐỀ TÀI:

PHÁT HIỆN KHẨU TRANG (MASK DETECTION)

Giảng Viên Hướng Dẫn:
Người Thực Hiện:

ThS. Phan Hồng Trung
22300659 - Nguyễn Hồng Đăng
22301260 - Dương Tấn Phát
22302421 - Nguyễn Phúc Minh Châu

Thành phố Hồ Chí Minh, ngày 16 tháng 06 năm 2026

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa và sự bùng nổ của cuộc cách mạng công nghiệp lần thứ tư, Trí tuệ Nhân tạo (AI) nói chung và Học sâu (Deep Learning) nói riêng đã trở thành một trong những công cụ cốt lõi, định hình lại cấu trúc công nghệ và thay đổi mạnh mẽ cách thức vận hành của nhiều lĩnh vực trong đời sống xã hội. Đặc biệt, phân ngành thị giác máy tính dựa trên các kiến trúc mạng thần kinh nhân tạo tiên tiến đã và đang mang lại những giải pháp tự động hóa với độ chính xác vượt trội, được ứng dụng rộng rãi từ y tế, xe tự lái cho đến các hệ thống giám sát an ninh thông minh. Nhận thức được tầm quan trọng mang tính thời đại cùng tiềm năng ứng dụng thực tiễn to lớn đó, nhóm chúng em đã quyết định lựa chọn và thực hiện đề tài "Xây dựng hệ thống nhận diện khuôn mặt và trạng thái đeo khẩu trang" làm nội dung nghiên cứu cho đề án môn học Deep Learning.

Báo cáo này là kết quả của quá trình nghiên cứu nghiêm túc, tập trung vào việc hiện thực hóa các lý thuyết nền tảng về mạng nơ-ron tích chập (CNN) và kỹ thuật học chuyển giao (Transfer Learning) vào một bài toán phân loại hình ảnh thực tế với sáu nhãn danh mục chi tiết. Không dừng lại ở việc tối ưu hóa các chỉ số trên tập dữ liệu thử nghiệm tĩnh, đề án còn đi sâu phân tích, đánh giá những thách thức mang tính kỹ thuật khi triển khai mô hình vào môi trường thực tế qua camera động như vấn đề nhiễu sáng, độ trễ và nhiễu hạt hệ thống. Qua đây, chúng em xin bày tỏ lòng biết ơn chân thành đến thầy Trung đã tận tình hướng dẫn, định hướng tư duy và tạo điều kiện thuận lợi nhất để nhóm có thể hoàn thành trọn vẹn đề án cũng như bồi dưỡng thêm kiến thức cho chúng em về môn học này.

MỤC LỤC

LỜI MỞ ĐẦU.....	1
MỤC LỤC.....	2
DANH MỤC HÌNH ẢNH & BẢNG.....	3
Bảng.....	3
Hình Ảnh.....	3
BẢNG PHÂN CÔNG VIỆC.....	4
NỘI DUNG.....	5
1. Khái Quát Vấn Đề.....	5
2. Mô Tả Bài Toán.....	5
3. Dataset.....	6
4. Tiền Xử Lý Dữ Liệu.....	7
5. Mô Hình.....	8
6. Kết Quả.....	10
7. Demo.....	11
7.1. Demo Phân Loại Hình Ảnh Mới.....	11
7.2. Demo Trên Camera.....	12
TÀI LIỆU THAM KHẢO.....	14

DANH MỤC HÌNH ẢNH & BẢNG

Bảng

Table 1: Bảng Phân Công Việc của Nhóm Thực Hiện.....	4
Table 2: Bảng Label Đầu Ra của Mô Hình.....	6
Table 3: Bảng Số Liệu Thông Tin Dataset.....	6

Hình Ảnh

Figure 1: Các nhãn được định dạng để ánh xạ với các annotation của dataset nằm ở folder.....	7
Figure 2: Nhãn dựa vào vị trí cấu trúc thư mục (folder-as-label) của Dataset.....	7
Figure 3: Cấu trúc thư mục để gọi các tệp dữ liệu có trong dataset.....	7
Figure 4: hàm kiểm tra cấu trúc thư mục và đường dẫn của dataset.....	7
Figure 5: Hàm tải và tiền xử lý hình ảnh.....	7
Figure 6: Chuyển đổi ngược ánh xạ annotation để dễ tính classification / confuse matrix.....	8
Figure 7: Sử dụng train_test_split để chia tệp dữ liệu thành tập train và validation.....	8
Figure 8: Cấu trúc chi tiết các tầng của mạng MobileNetV2.....	8
Figure 9: Cấu trúc mô hình của MobileNetV2.....	8
Figure 10: Bảng so sánh giữa Train mô hình CNN từ Scratch và Transfer Learning.....	9
Figure 11: Model Summary.....	9
Figure 12: Workflow của Mô Hình Huấn Luyện.....	9
Figure 13: Kết quả đánh giá và báo cáo phân loại (Classification Report) trên tập Validation....	10
Figure 14: Plot Model Classification Accuracy.....	10
Figure 15: Plot Model Cross-Entropy Loss.....	11
Figure 16: Confusion Matrix.....	11
Figure 17: Hàm tải mô hình và ánh xạ nhãn.....	11
Figure 18: Hàm dự đoán và hiển thị kết quả Demo.....	12
Figure 19: Kết quả Without Mask (Simple).....	12
Figure 20: Kết quả Without Mask (Complex).....	12
Figure 21: Kết quả With Mask (Simple).....	12
Figure 22: Demo Ứng Dụng Nhận Diện Khẩu Trang.....	13

BẢNG PHÂN CÔNG VIỆC

MSSV	Họ và Tên	Công Việc	Mức Độ Hoàn Thành
22300659	Nguyễn Hồng Đăng	- Code mô hình huấn luyện - Thiết kế UI Demo	100%
22302421	Nguyễn Phúc Minh Châu	- Viết Báo Cáo Đồ Án - Thực hiện patch & fix code	100%
22301260	Dương Tấn Phát	- Lập bản thuyết trình - Demo Mô Hình	100%

Table 1: Bảng Phân Công Việc của Nhóm Thực Hiện

NỘI DUNG

1. Khái Quát Vấn Đề

Vào cuối năm 2019 - đầu năm 2020, cả thế giới bùng nổ vì một đại dịch nguy hiểm đến từ loài dơi với các triệu chứng không khác gì với việc cảm lạnh thông thường nhưng lại mang tới một tỉ lệ tử vong cao là SARS-CoV-2 hay còn biết đến với tên gọi là Covid-19. Với tính chất lan truyền dịch bệnh dễ dàng qua giọt bắn đường hô hấp khi tiếp xúc gần nên khi đây Tổ chức Y Tế (WHO) khuyến nghị và chính phủ các nước áp dụng bắt buộc quy định người dân phải đeo khẩu trang tại nơi công cộng để giảm thiểu sự lây truyền bệnh tăng nhanh song song thực hiện cách ly để kiểm soát dịch bệnh.

Dù vậy ở những nơi đông người như các trạm y tế, bệnh viện, trường học, sân bay, v.v... rất khó để kiểm soát thủ công và tốn nhiều nguồn nhân lực cũng như chi phí để giám sát và yêu cầu mọi người có thể tuân thủ quy định an toàn và đeo khẩu trang nơi công cộng. Với điều kiện đây một phần mềm hoặc thiết bị có thể phát hiện người đeo khẩu trang là cần thiết (vào lúc đó còn có thể tích hợp tính năng đo thân nhiệt và khử khuẩn tay).

2. Mô Tả Bài Toán

Bài toán được thực hiện là **phân loại hình ảnh (image classification)** để huấn luyện cho mô hình có thể nhận biết được người và khẩu trang trên mặt người. Sau đó sử dụng kết quả mô hình đã được huấn luyện để nhận diện mặt người thật và phân loại bằng camera (hình ảnh chạy chụp liên tục).

Đầu ra của bài toán sẽ in ra kết quả nhận dạng của mô hình trong đó có sáu nhãn phân loại sau:

Tên Nhãn	Đặc Trưng	Chú Thích
Incorrect Mask (MC) (Mouth Covered)	- Khẩu trang chỉ che phần miệng - Vùng mũi lộ hoàn toàn - Góc chụp trực diện, ánh sáng rõ ràng	Đeo sai quy định. Hệ thống phát cảnh báo nhắc nhở chỉnh lại khẩu trang
Incorrect Mask (MMC) (Mouth/Nose Miscovered)	- Khẩu trang bị kéo xuống cằm hoặc đeo lệch - Lộ cả mũi lẫn miệng - Góc chụp khó (nghiêng khuất) hoặc ánh sáng yếu	Đeo sai quy định. Tương đương với việc không đeo. Hệ thống sẽ phát cảnh báo không thể nhận dạng rõ ràng.
With Mask (Complex)	- Khẩu trang che kín từ sống mũi xuống cằm - Bị che khuất một phần bởi tóc, kính, nón hoặc người dùng di chuyển, góc chụp tối	Hệ thống sẽ đánh giá là hợp lệ. Mô hình vượt qua độ nhiễu để xác nhận an toàn
With Mask (Simple)	- Khẩu trang đeo chuẩn, vuông vắn - Khuôn mặt nhìn thẳng vào camera	Hệ thống đánh giá là an toàn lý tưởng có thể nhận diện nhanh

	Ảnh sáng phòng studio/văn phòng tiêu chuẩn	
Without Mask (Complex)	- Toàn bộ khuôn mặt lộ rõ (không đeo khẩu trang) - Khuôn mặt bị quay nghiêng, cúi xuống hoặc có vật cản che mặt một phần (ví dụ: đưa tay lên mặt)	Hệ thống sẽ cảnh báo vi phạm. Phát hiện người dùng cố tình né tránh camera hoặc trong bối cảnh phức tạp.
Without Mash (Simple)	- Toàn bộ khuôn mặt lộ rõ (không đeo khẩu trang) - Nhìn thẳng vào camera thấy rõ ngũ quan - Không có bất kỳ vật cản nào	Hệ thống sẽ cảnh báo vi phạm

Table 2: Bảng Label Đầu Ra của Mô Hình

Với số nhãn nhiều hơn so với yêu cầu của đề bài phần nào muốn mô hình đánh giá phù hợp thực tế trong bối cảnh dịch bệnh tại các nơi ra vào công cộng. Giúp mô hình xác định được các lỗi vi phạm, nhiều hình ảnh cũng như nâng độ chính xác trong nhận diện việc đeo khẩu trang trên mặt người.

3. Dataset

Trong đề án này nhóm sử dụng [Face Mask Dataset](#) được Shiekh Burhan đăng tải công khai trên Kaggle (16/06/2022), gắn liền với nghiên cứu RRFMDs (Rapid Real-Time Face Mask Detection System) phục vụ giám sát tuân thủ đeo khẩu trang trong đại dịch COVID-19.

Dữ liệu được tổng hợp từ nhiều nguồn: các bộ dữ liệu chuẩn đã công bố trước đó, các kho ảnh trên internet, và dữ liệu mô phỏng (simulated data) nhằm đảm bảo tính đa dạng và giảm thiên lệch (unbiased representation). Mục đích ban đầu là dùng để huấn luyện một mô hình phân loại khẩu trang (face mask classifier), kết hợp với một bộ tách khuôn mặt (MTCNN/SSD) trong pipeline phát hiện thời gian thực.

Dataset có 3 lớp và 6 lớp phụ phân loại, tương ứng với 3 trạng thái đeo khẩu trang:

Class Label	Images Amount	Sub Label
With Mask	4.789	Simple, Complex
Without Mask	4.746	Simple, Complex
Incorrect Mask	5.000	MC, MMC
Sum Images Amount:	14.535	

Table 3: Bảng Số Liệu Thông Tin Dataset

Số annotation tương đương với **nhãn lớp (class label) gắn theo từng ảnh** — thường được tổ chức dưới dạng thư mục con (mỗi thư mục = một nhãn), không có file annotation định dạng XML/JSON kiểu COCO hay YOLO.

4. Tiền Xử Lý Dữ Liệu

Vì Dataset không có file annotation rời (XML/JSON/CSV) mà dùng **annotation ngầm định qua cấu trúc thư mục** (folder-as-label). Hàm `load_dataset_paths()` quét 6 thư mục con cố định:

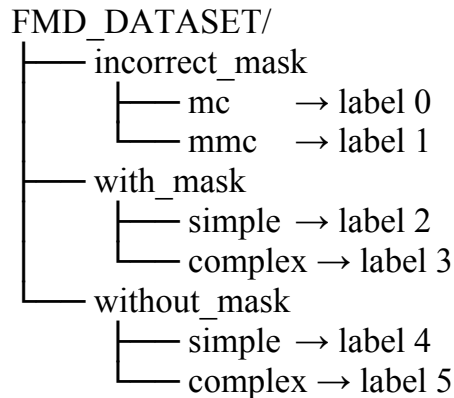


Figure 2: Nhãn dựa vào vị trí cấu trúc thư mục (folder-as-label) của Dataset

```

CLASS_MAPPING = {
    "incorrect_mask_mc": 0,
    "incorrect_mask_mmc": 1,
    "with_mask_complex": 2,
    "with_mask_simple": 3,
    "without_mask_complex": 4,
    "without_mask_simple": 5
}
  
```

Figure 1: Các nhãn được định dạng để ánh xạ với các annotation của dataset nằm ở folder

```

structure = {
    "incorrect_mask/mc": "incorrect_mask_mc",
    "incorrect_mask/mmc": "incorrect_mask_mmc",
    "with_mask/complex": "with_mask_complex",
    "with_mask/simple": "with_mask_simple",
    "without_mask/complex": "without_mask_complex",
    "without_mask/simple": "without_mask_simple"
}
  
```

Figure 3: Cấu trúc thư mục để gọi các tệp dữ liệu có trong dataset

Mỗi ảnh hợp lệ (.jpg .jpeg .png .bmp .webp) được gán nhãn số nguyên theo `CLASS_MAPPING`, trả về 2 mảng numpy: `paths` (đường dẫn ảnh) và `labels` (nhãn tương ứng). Việc đọc pixel thật sự diễn ra sau, trong `load_and_preprocess_image()`, dùng `tf.io.read_file + tf.image.decode_jpeg` (lazy loading qua `tf.data`, không load hết RAM cùng lúc).

```

def check_and_prepare_dirs():
    """Validates dataset existence and creates output directories."""
    if not DATASET_DIR.exists() or not DATASET_DIR.is_dir():
        print(f"[ERROR] Dataset directory '{DATASET_DIR}' does not exist.")
        print("Please ensure the dataset structure matches the requirements.")
        sys.exit(1)

    # Create output directory if it doesn't exist
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
  
```

Figure 4: hàm kiểm tra cấu trúc thư mục và đường dẫn của dataset

```

def load_and_preprocess_image(path, label):
    """tf.data pipeline function to read, decode, and preprocess images."""
    image = tf.io.read_file(path)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, IMAGE_SIZE)
    # MobileNetV2 preprocess input standardizes pixels to [-1, 1] range
    image = keras.applications.mobilenet_v2.preprocess_input(image)
    return image, label
  
```

Figure 5: Hàm tải và tiền xử lý hình ảnh

Vì không có annotation gốc dạng VOC/COCO/YOLO để convert. Chuyển đổi duy nhất xảy ra ở đây là **string to integer label** thông qua dictionary `CLASS_MAPPING`, và ngược lại bằng `REVERSE_CLASS_MAPPING` (dùng khi in classification report / confusion matrix). Ảnh thì được normalize qua `mobilenet_v2.preprocess_input` (đưa pixel về khoảng `[-1, 1]` đúng chuẩn input MobileNetV2).

```
CLASS_MAPPING = {
    "incorrect_mask_mc": 0,
    "incorrect_mask_mmc": 1,
    "with_mask_complex": 2,
    "with_mask_simple": 3,
    "without_mask_complex": 4,
    "without_mask_simple": 5
}
REVERSE_CLASS_MAPPING = {v: k for k, v in
    CLASS_MAPPING.items()}
NUM_CLASSES = len(CLASS_MAPPING)
```

Figure 6: Chuyển đổi ngược ảnh xạ annotation để dễ tính classification / confuse matrix

```
train_paths, val_paths, train_labels, val_labels =
train_test_split
(
    paths,
    labels,
    test_size=0.20,
    stratify=labels,
    random_state=RANDOM_STATE
)
```

Figure 7: Sử dụng `train_test_split` để chia tập dữ liệu thành tập train và validation

Sau khi đã scan được hết các tệp ảnh trong dataset, sử dụng `train_test_split` của scikit-learn, **stratified split** theo nhãn (`stratify=labels`) để giữ tỉ lệ 6 lớp đồng đều giữa hai tập: **Train: 80%, Validation: 20%**

5. Mô Hình

Mô hình được sử dụng là Transfer Learning với **MobileNetV2** (pretrained trên ImageNet) làm backbone trích xuất đặc trưng, đóng băng toàn bộ (`base_model.trainable = False`). **MobileNetV2** được thiết kế cho thiết bị tài nguyên hạn chế (depthwise separable convolutions, inverted residuals) — phù hợp vì mục tiêu cuối là chạy real-time qua camera, không phải chỉ chạy offline. Việc đóng băng backbone nhằm giảm số lượng tham số cần học, giảm overfitting, tăng tốc huấn luyện (có thể fine-tune unfreeze sau nếu muốn tăng accuracy).

Input	Operator	<i>t</i>	<i>c</i>	<i>n</i>	<i>s</i>
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	-

Figure 8: Cấu trúc chi tiết các tầng của mạng MobileNetV2

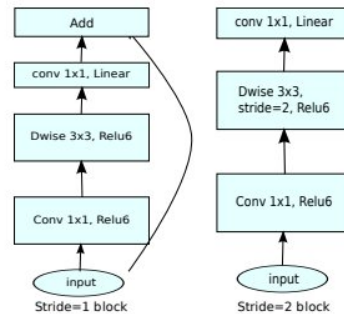


Figure 9: Cấu trúc mô hình của MobileNetV2

Transfer Learning từ ImageNet giúp tận dụng đặc trưng thị giác tổng quát (cạnh, texture, hình dạng) đã học từ triệu ảnh, rất hiệu quả khi dataset không đủ lớn để train from scratch.

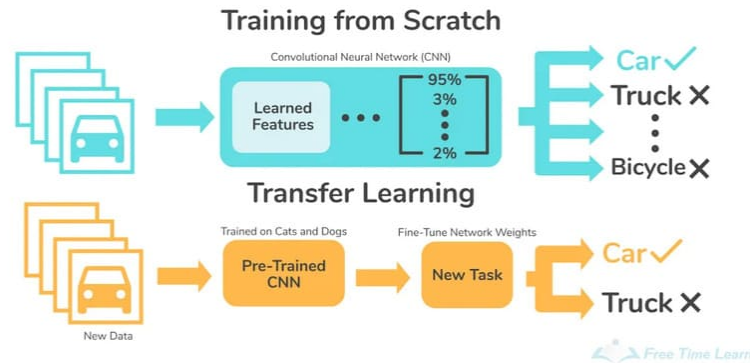


Figure 10: Bảng so sánh giữa Train mô hình CNN từ Scratch và Transfer Learning

Pipeline Huấn Luyện Mô Hình:

Model: "FMO_MobileNetV2_Classifier"		
Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense (Dense)	(None, 256)	327,936
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 6)	1,542
Total params: 2,587,462 (9.87 MB)		
Trainable params: 329,478 (1.26 MB)		
Non-trainable params: 2,257,984 (8.61 MB)		

Figure 11: Model Summary

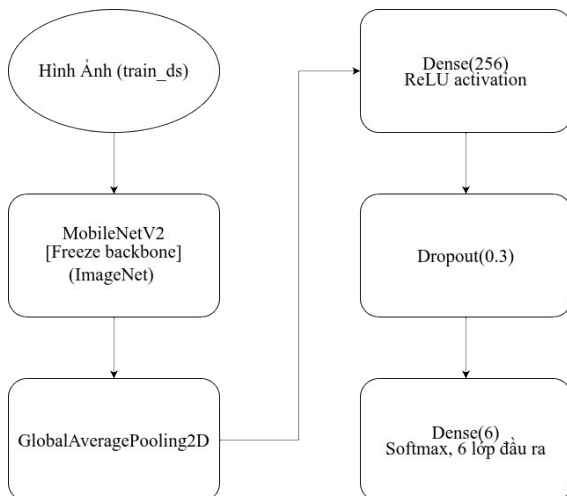


Figure 12: Workflow của Mô Hình Huấn Luyện

Song song với `train_ds`, sau mỗi epoch model chạy forward-pass (không backprop) trên `val_ds` để tính `val_loss/val_accuracy` — đây là input cho 3 callback:

- **ModelCheckpoint:** so sánh `val_loss` epoch hiện tại với best đã lưu → nếu tốt hơn, ghi đè `best_model.keras`.
- **EarlyStopping:** theo dõi `val_loss` liên tiếp 3 epoch, nếu không cải thiện → dừng training, rollback về weight tốt nhất (`restore_best_weights=True`).
- **ReduceLROnPlateau:** theo dõi `val_loss` liên tiếp 2 epoch, nếu plateau → nhân learning rate hiện tại với 0.2.

Vòng lặp này lặp lại tối đa 50 epoch hoặc dừng sớm hơn nếu `EarlyStopping` kích hoạt để tránh overfitting. Vì backbone bị đóng băng, số tham số thực sự được cập nhật trong suốt pipeline chỉ nằm ở phần head (`Dense 256` → `Dropout` → `Dense 6`), giúp việc huấn luyện nhẹ và nhanh hơn so với train toàn bộ `MobileNetV2`. Cuối cùng sẽ lưu model cuối (`final_model.keras`) + model tốt nhất (`best_model.keras`).

6. Kết Quả

Sau khi huấn luyện xong mô hình, kết quả của bản Final Validation cho thấy độ chính xác cấu trúc đạt 92.74% và hàm mất mát cực thấp ở mức 0.2080. Nhìn chung, mô hình thể hiện khả năng nhận diện xuất sắc nhất ở các bối cảnh đơn giản nhờ chỉ số F1-score đạt từ 0.95 đến 0.97, với lớp "without_mask_simple" đạt độ chính xác (Precision) ở mức 98%.

```
Final Validation Dataset Results:
- Validation Loss: 0.2080
- Validation Accuracy: 92.74%

Classification Report:
```

	precision	recall	f1-score	support
incorrect_mask_mc	0.88	0.95	0.91	500
incorrect_mask_mmc	0.91	0.86	0.89	500
with_mask_complex	0.87	0.76	0.81	158
with_mask_simple	0.93	0.97	0.95	800
without_mask_complex	0.91	0.77	0.83	150
without_mask_simple	0.98	0.97	0.97	800
accuracy			0.93	2908
macro avg	0.91	0.88	0.90	2908
weighted avg	0.93	0.93	0.93	2908

Figure 13: Kết quả đánh giá và báo cáo phân loại (Classification Report) trên tập Validation.

Đối với các lớp liên quan đến việc đeo khẩu trang sai cách, lớp "incorrect_mask_mc" có độ phủ (Recall) đạt 95%, đồng nghĩa với việc mô hình rất ít khi bỏ sót trường hợp này, dù đôi khi có sự nhầm lẫn nhẹ khiến Precision dừng ở mức 88%. Ở các lớp có mức độ complex, nơi chỉ số Recall của "with_mask_complex" và "without_mask_complex" lần lượt hạ xuống còn 76% và 77% khiến mô hình khó nhận diện cả trường hợp có độ nhiễu cao.

Dù vậy, Macro F1-score đạt 0.90 và Weighted F1-score đạt 0.93 trên tổng quy mô 2908 mẫu thử nghiệm, mô hình hoàn toàn đáp ứng tốt tiêu chuẩn phân loại đáng tin cậy trong các ứng dụng giám sát thực tế.

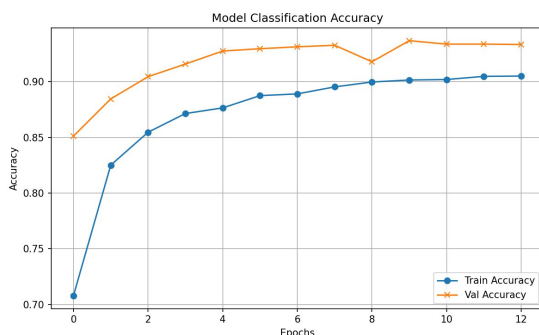


Figure 14: Plot Model Classification Accuracy

Model Classification Accuracy cho thấy đầu tiên là với tối đa 50 epoch được huấn luyện, mô hình đã dừng lại ở epoch thứ 12 nhờ vào EarlyStopping để cho mô hình tránh bị Overfitting.

Plot cho thấy mức độ chính xác của mô hình khi chạy trên tập dữ liệu train và validation, nhìn chung mô hình có tốc độ học rất nhanh ở các epochs đầu tiên sau đấy tăng dần.

Khoảng cách giữa hai đường tiệm cận sát nhau ở giai đoạn cuối, cho thấy mô hình đạt được độ hội tụ tốt với độ chính xác tổng thể duy trì ổn định trên mức 90%.

Model Cross-Entropy Loss cho thấy mức độ tỉ lệ nghịch tương ứng với **Model Classification Accuracy** khi độ chính xác càng tăng thì độ mất mát càng giảm. Tập train bắt đầu với mức loss mức 0.82 nhưng nhanh chóng giảm dần xuống khoảng 0.27 ở epoch cuối cùng. Đường loss của tập validation cũng giảm đều từ 0.42 xuống khoảng 0.20.

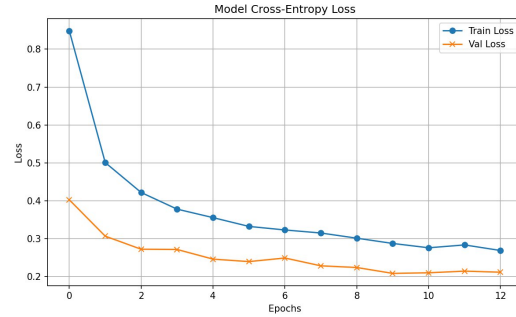


Figure 15: Plot Model Cross-Entropy Loss

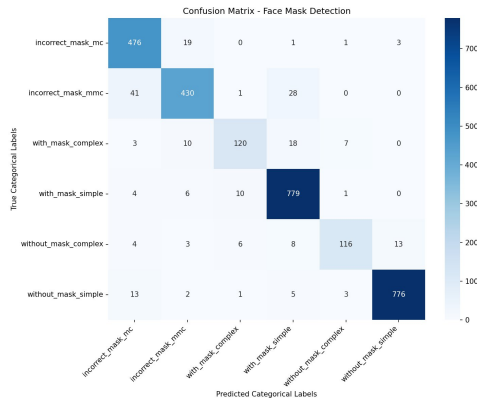


Figure 16: Confusion Matrix

Confusion Matrix cho thấy các giá trị nằm trên đường chéo chính chiếm tỷ lệ cao, chứng minh độ chính xác cao của mô hình. **Confusion Matrix** cũng cho thấy sự nhầm lẫn xảy ra không đáng kể và chủ yếu tập trung giữa các biến thể có cùng bản chất nền tảng xuất hiện giữa môi trường bối cảnh phức tạp (complex) và đơn giản (simple), hoặc giữa việc đeo sai cách và đeo đúng cách.

7. Demo

7.1. Demo Phân Loại Hình Ảnh Mới

Khi cho thử nghiệm mô hình với một số hình ảnh khác bằng cách quan sát trên Jupiter Notebook, bằng cách tải về mô hình đã được huấn luyện trước đó rồi ánh xạ với 6 label có trong tập huấn luyện cùng một ảnh mới không có trong dataset để mô hình phân loại.

```
MODEL_PATH = "best_model.keras"
IMAGE_PATH = "03.jpg"

print("Đang tải mô hình, vui lòng đợi...")
model = keras.models.load_model(MODEL_PATH)
print("Tải mô hình thành công!")

class_mapping = {
    0: "Incorrect Mask (MC)",
    1: "Incorrect Mask (MMC)",
    2: "With Mask (Complex)",
    3: "With Mask (Simple)",
    4: "Without Mask (Complex)",
    5: "Without Mask (Simple)"
}

class_colors = {
    0: "orange",
    1: "darkorange",
    2: "green",
    3: "limegreen",
    4: "red",
    5: "darkred"
}
```

Figure 17: Hàm tải mô hình và ánh xạ nhãn

```
def predict_and_display(image_path, ai_model):
    img = cv2.imread(image_path)
    if img is None:
        print(f"[Lỗi] Không tìm thấy hoặc không thể đọc ảnh: {image_path}")
        return

    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    resized_img = cv2.resize(img_rgb, (224, 224))

    preprocessed_img = keras.applications.mobilenet_v2.preprocess_input(resized_img.astype(np.float32))
    input_tensor = np.expand_dims(preprocessed_img, axis=0)

    predictions = ai_model.predict(input_tensor, verbose=0)[0]
    class_idx = np.argmax(predictions)
    confidence = predictions[class_idx]
    label = class_mapping[class_idx]
    color = class_colors[class_idx]

    plt.figure(figsize=(8, 8))
    plt.imshow(img_rgb)

    title_text = f"Kết quả: {label} | Độ tự tin: {confidence*100:.2f}%"
    plt.title(title_text, color=color, fontsize=16, fontweight='bold', pad=15)

    plt.axis('off')
    plt.tight_layout()
    plt.show()

predict_and_display(IMAGE_PATH, model)
```

Figure 18: Hàm dự đoán và hiển thị kết quả Demo

Ảnh gốc sau khi được đọc bởi OpenCV sẽ chuyển đổi sang gradient trước khi thay đổi kích thước ma trận pixel về độ phân giải cố định. Sau khi ép kiểu dữ liệu sang dạng số thực và chạy qua hàm tiền xử lý đặc trưng, mảng dữ liệu được mở rộng thêm một chiều batch để tạo thành tensor đầu vào hợp lệ với mô hình. Dự đoán sẽ cho ra nhãn có độ tự tin cao nhất.

Có thể thấy rõ là mô hình đã học rất tốt khi được huấn luyện, khi được tiếp xúc với các ảnh mới đều có mức độ chính xác trên 80%.



Figure 19: Kết quả Without Mask (Simple)

Figure 20: Kết quả Without Mask (Complex)

Figure 21: Kết quả With Mask (Simple)

7.2. Demo Trên Camera

Khi thử nghiệm trên tập dữ liệu tĩnh hoặc các hình ảnh mới, mô hình dễ dàng đưa ra dự đoán chính xác với độ tự tin cao trên 80%. Tuy nhiên, trong môi trường động như nhận diện trực tiếp qua camera, hiệu suất của hệ thống bị suy giảm đáng kể do các yếu tố nhiễu từ ánh sáng biến động và bối cảnh xung quanh. Thêm vào đó, việc sử dụng thư viện OpenCV để bắt luồng video cũng phát sinh các vấn đề về độ trễ, nhiễu hạt và sự sai lệch trong kích thước ảnh đầu vào, gây ảnh hưởng trực tiếp đến độ chính xác.

Ngược lại, nhờ việc định nghĩa đa dạng các nhãn phân loại (label) chi tiết trong quá trình huấn luyện, mô hình đã tối ưu hóa khả năng nhận diện tốt, giảm thiểu tối đa hiện tượng sai lệch khi khuôn mặt người dùng bị che khuất một phần bởi tay hoặc vật cản.

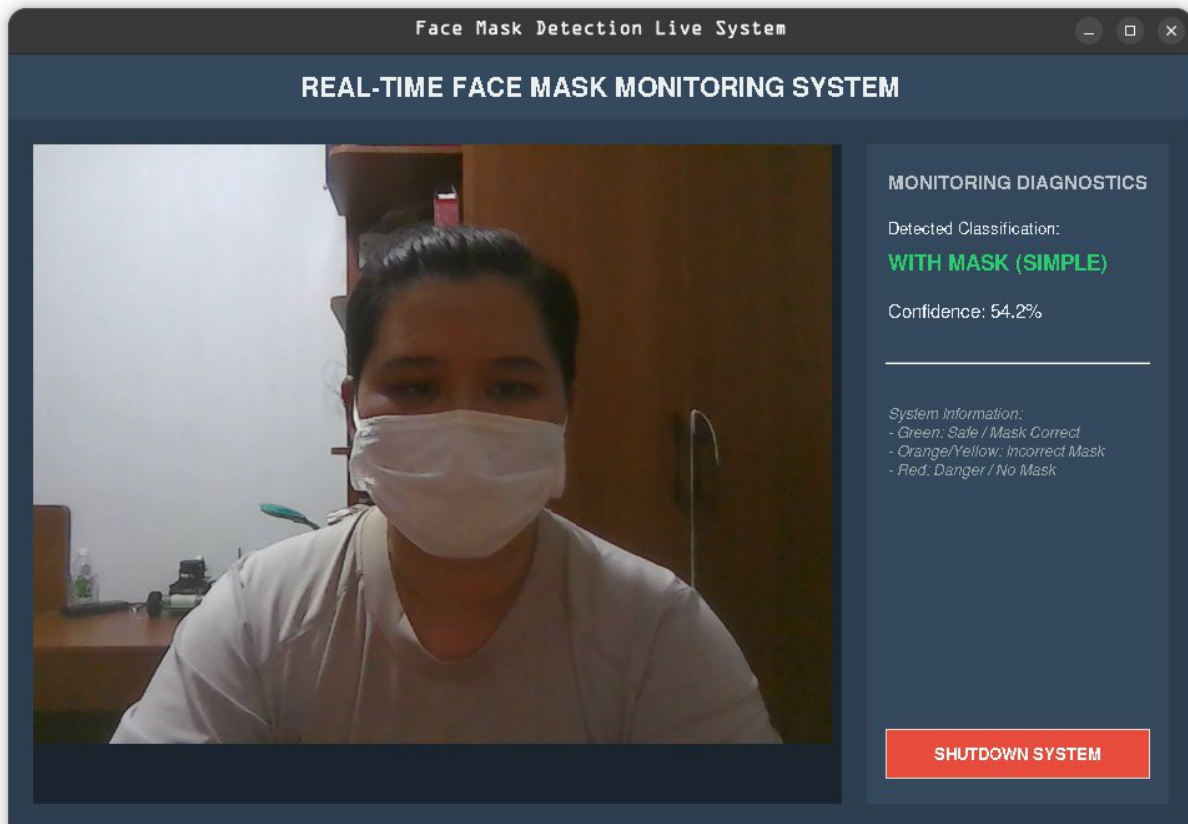


Figure 22: Demo Ứng Dụng Nhận Diện Khẩu Trang

TÀI LIỆU THAM KHẢO

Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications.

Pattanayak, S. (2021). *Pro deep learning with TensorFlow 2.0: A mathematical approach to advanced artificial intelligence in Python* (2nd ed.). Apress

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4510–4520. <https://arxiv.org/abs/1801.04381>